

SKILL: taller-bv-dashboard-tech

Código exacto para extraer, aplicar y validar cambios en el Dashboard Taller BV.

Rutas

- Dashboard (bash VM): /sessions/*/mnt/Taller BV/Dashboard_Pedidos.html
- XLSX descargado: /tmp/urgentes.xlsx
- Output subagente: /sessions/*/mnt/outputs/extraccion_resultado.json

Decodificar XLSX desde base64

```
import json, base64
with open('<ruta_del_txt>') as f:
    data = json.load(f)
raw = base64.b64decode(data['content'])
with open('/tmp/urgentes.xlsx', 'wb') as f:
    f.write(raw)
# Verificar: raw[:2] == b'PK' (es ZIP/XLSX)
```

Extraer producción por GA (openpyxl) — DETECCIÓN AUTOMÁTICA DE MES

Regla de meses (importante):

- El mes se detecta por el **texto del rótulo** objetivo <Mes> en la fila 1 y por las columnas de fecha (fila 2) de ese mes/año. NO hardcodear el mes.
 - Sincronizar SIEMPRE: el **mes en curso** (fecha AR) + cualquier **mes futuro ya armado** en el sheet (ej. si ya cargaron las columnas de agosto).
 - NUNCA re-sincronizar meses históricos ya congelados en el dashboard (los que son anteriores al mes en curso): se dejan tal cual.
 - Un mes con columnas de fecha pero SIN objetivo cargado se incluye igual, como bloque "vacío": obj_mensual=0, semanas=[], codigos=[], prod_total=0.
- Cuando en el sheet se cargue el objetivo/producción, un sync posterior lo completa. (semanas=[] evita el NaN en calcProd del dashboard.)
- Estatores: ignorar la columna con fecha 30/04. No contar filas con código "-"/vacío.
 - Los objetivos semanales del sheet vienen decimales → redondear al entero.

```
import openpyxl
from datetime import datetime, timezone, timedelta
from collections import defaultdict

wb = openpyxl.load_workbook('/tmp/urgentes.xlsx', data_only=True)

MESES_ES = {1:'Enero',2:'Febrero',3:'Marzo',4:'Abril',5:'Mayo',6:'Junio',
            7:'Julio',8:'Agosto',9:'Septiembre',10:'Octubre',11:'Noviembre',12:'Diciembre'}

def extraer_ga(sheet_name, year, month):
    """Bloque de producción de un mes/año. Devuelve None solo si el mes NO está
    armado en el sheet (no hay columnas de fecha de ese mes)."""
    ws = wb[sheet_name]
    mes_nombre = MESES_ES[month]
    maxc = ws.max_column
    row1 = {c: ws.cell(1, c).value for c in range(1, maxc+1)}

    # localizar la celda 'Objetivo <Mes>' (tolera ':' final y mayúsc/minúsc)
    lab_c = None
    for c in range(1, maxc+1):
        v = row1[c]
        if isinstance(v, str) and v.strip().rstrip(':').lower() == ('objetivo '+mes_nombre).lower():
            lab_c = c; break
    obj_mensual = row1.get(lab_c+1) if lab_c else None

    # objetivos semanales del bloque: van ANTES del label mensual (dentro de las
```

```

# columnas de fecha del mes), entre el 'Objetivo' del mes anterior y el de este mes.
# OJO: NO escanear hacia adelante (agarraría los del mes siguiente).
obj_sems = {}
if lab_c:
    prev = 0
    for c in range(lab_c-1, 0, -1):
        v = row1.get(c)
        if isinstance(v, str) and v.strip().lower().startswith('objetivo'):
            prev = c; break
    for c in range(prev+1, lab_c):
        v = row1.get(c)
        if isinstance(v, str) and 'Obj. S.' in v:
            try:
                num = int(v.split('.')[0].strip()); val = row1.get(c+1)
                if isinstance(val, (int, float)): obj_sems[num] = round(val) # ENTERO
            except: pass

# columnas de fecha del mes/año (Estatores: ignorar 30/04)
cols = []
for c in range(1, maxc+1):
    v = ws.cell(2, c).value
    if isinstance(v, datetime) and v.year == year and v.month == month:
        if sheet_name == 'OF-EST' and v.month == 4 and v.day == 30: continue
        cols.append((c, v))
if not cols:
    return None # el mes no está armado en el sheet

# producción total (EXCLUIR filas con código '-' o vacío = son totales)
total = 0; codigos = []
for row in range(3, ws.max_row+1):
    cod = ws.cell(row, 1).value
    if not cod or not isinstance(cod, str): continue
    cod = cod.strip()
    if cod in ('-', ''): continue
    p = sum(ws.cell(row,c).value or 0 for c,_ in cols
            if isinstance(ws.cell(row,c).value, (int,float)))
    if p > 0:
        cat = ws.cell(row, 2).value; total += p
        codigos.append({'codigo': cod, 'categoria': str(cat).strip() if cat else '',
                        'cantidad': int(p)})

obj_m = round(obj_mensual) if isinstance(obj_mensual, (int, float)) else 0

# Mes armado pero SIN objetivo ni producción -> bloque vacío (semanas=[] evita NaN)
if obj_m == 0 and total == 0:
    return {'mes_key': f'{year:04d}-{month:02d}', 'mes_label': f'{mes_nombre} {year}',
            'obj_mensual': 0, 'dias_habiles': len(cols),
            'prod_total': 0, 'semanas': [], 'codigos': []}

# semanas (agrupadas por número de semana ISO)
sem_map = defaultdict(list)
for c, dt in sorted(cols, key=lambda x: x[1]):
    sem_map[dt.isocalendar()[1]].append((c, dt))
semanas = []
for i, (wk, ccols) in enumerate(sorted(sem_map.items())):
    prod_s = 0
    for row in range(3, ws.max_row+1):
        cod = ws.cell(row,1).value
        if not cod or not isinstance(cod, str) or cod.strip() in ('-', ''): continue
        prod_s += sum(ws.cell(row,c).value or 0 for c,_ in ccols
                      if isinstance(ws.cell(row,c).value, (int,float)))
    semanas.append({'num': i+1, 'dias': len(ccols),
                    'obj': obj_sems.get(i+1, 0), 'prod': int(prod_s)})

return {'mes_key': f'{year:04d}-{month:02d}', 'mes_label': f'{mes_nombre} {year}',
        'obj_mensual': obj_m, 'dias_habiles': len(cols),
        'prod_total': int(total), 'semanas': semanas, 'codigos': codigos}

# --- Meses a sincronizar: mes en curso (AR) + meses futuros ya armados ---
hoy = datetime.now(timezone(timedelta(hours=-3)))
objetivo_meses = [(hoy.year, hoy.month)]
for m in range(hoy.month + 1, 13):
    if extraer_ga('OF-IND', hoy.year, m) is not None:
        objetivo_meses.append((hoy.year, m))

bloques = {} # mes_key -> {ga: bloque}
for (Y, M) in objetivo_meses:
    for ga_name, sheet in [('Inducidos', 'OF-IND'), ('Rotores', 'OF-ROT'), ('Estatores', 'OF-EST')]:
        r = extraer_ga(sheet, Y, M)
        if r is None: continue
        bloques.setdefault(r['mes_key'], {})[ga_name] = r

```

Al aplicar al HTML: para cada mes_key en bloques, buscar la entrada de OF_DATA con ese mes_key + ga. Si existe, actualizar obj_mensual, prod_total, semanas, codigos, dias_habiles. Si NO existe (mes nuevo, ej. agosto), agregar la entrada. Ordenar OF_DATA por mes_key y luego por GA (Inducidos, Rotores, Estatores). Nunca tocar meses que no estén en bloques (históricos congelados).

Extraer pedidos (hoja Urgentes)

```
ws = wb['Urgentes']
# Col 1=N, 2=Ingreso, 3=GA, 4=Cod, 5=Cond, 6=Cliente
# Col 7=Días obj, 8=Entrega sol, 12=Reclamo, 13=Obs
# Col 14=Entrega real, 16=Días prod. (P), 17=Demora (Q)

def fmt_date(v):
    if isinstance(v, datetime): return v.strftime('%Y-%m-%d')
    if isinstance(v, str) and v not in ('-', '.'): return v
    return None

pedidos_sheet = {}
for row in range(3, ws.max_row+1):
    n = ws.cell(row,1).value; ga = ws.cell(row,3).value
    if not isinstance(n,(int,float)) or not isinstance(ga,str): continue
    n = int(n)
    dias_prod_raw = ws.cell(row,16).value
    # dias_prod negativo = artifact de formula -> null
    dias_prod = int(dias_prod_raw) if isinstance(dias_prod_raw,(int,float)) and dias_prod_raw >= 0 else None
    cod = ws.cell(row,4).value
    cod = str(cod).strip().replace(',','.') if cod else None # normalizar coma->punto en códigos
    pedidos_sheet[n] = {
        'n': n, 'ga': str(ga).strip(),
        'codigo': cod,
        'condicion': ws.cell(row,5).value, 'cliente': ws.cell(row,6).value,
        'fecha_ingreso': fmt_date(ws.cell(row,2).value),
        'dias_objetivo': int(ws.cell(row,7).value) if isinstance(ws.cell(row,7).value,(int,float)) else None,
        'entrega_sol': fmt_date(ws.cell(row,8).value),
        'reclamo': ws.cell(row,12).value if ws.cell(row,12).value not in (None,'-') else None,
        'observacion': ws.cell(row,13).value if ws.cell(row,13).value not in (None,'-') else None,
        'fecha_entrega_real': fmt_date(ws.cell(row,14).value),
        'dias_prod': dias_prod,
        'demora': round(float(ws.cell(row,17).value),4) if isinstance(ws.cell(row,17).value,(int,float)) else None,
    }
```

Actualizar badge

```
import re
from datetime import datetime, timezone, timedelta
AR = timezone(timedelta(hours=-3))
ahora = datetime.now(AR)
dias = ['Lun','Mar','Mié','Jue','Vie','Sáb','Dom']
fecha_str = f"{dias[ahora.weekday()]} {ahora.strftime('%d/%m/%Y %H:%M')}"
html = re.sub(r"const _ULTIMO_SYNC='[^']*'",
              f"const _ULTIMO_SYNC='{fecha_str}'", html)
# CRITICO: usar re.sub, NO html.find(";")
```

Serializador OF_DATA / PEDIDOS (JS-literal, mismo estilo del dashboard)

Al re-serializar los arrays, usar claves sin comillas, strings con comilla simple, null para vacíos, y CERRAR el bloque con]; (no olvidar el punto y coma):

```
const isId=k=>/^[A-Za-z_$][A-Za-z0-9_$]*$/ .test(k);
function ser(v){
    if(v===null||v===undefined) return 'null';
    if(typeof v==='number'||typeof v==='boolean') return String(v);
    if(typeof v==='string') return ""+v.replace(/\//g,'\\').replace(/'/g,"\\'")+"";
    if(Array.isArray(v)) return '['+v.map(ser).join(',')+']';
    if(typeof v==='object') return '{'+Object.entries(v).map(([k,val])=>(isId(k)?k:JSON.stringify(k))+':'+ser(val))).join(',')
}
// const OF_DATA=[\n  {...},\n  ...\\n]; <-- terminar con ];
```

Validación post-aplicación (Node.js)

```
node -e "
const fs=require('fs');
const html=fs.readFileSync('/sessions/*/mnt/Taller BV/Dashboard_Pedidos.html','utf8');
const ofStart=html.indexOf('const OF_DATA=');
const pedEnd=html.indexOf('];',html.indexOf('const PEDIDOS=')+2);
try {
  const {OF_DATA,PEDIDOS}=new Function(
    html.slice(ofStart,html.indexOf('];',ofStart)+2)+'\n'+
    html.slice(html.indexOf('const PEDIDOS='),pedEnd)+'\nreturn {OF_DATA,PEDIDOS};')();
  console.log('OF_DATA:',OF_DATA.length,'| PEDIDOS:',PEDIDOS.length);
  console.log('mes_keys:',[...new Set(OF_DATA.map(x=>x.mes_key))].join(', '));
  console.log('JS OK');
} catch(e){ console.log('ERROR:',e.message); }
```

No sobrescribir con vacíos

```
def es_cambio(campo, v_dash, v_sheet):
    if v_sheet is None: return False # no sobrescribir con vacio
    if v_dash == v_sheet: return False
    if campo == 'demora' and v_dash is not None:
        if round(float(v_dash),2) == round(float(v_sheet),2): return False
    if campo == 'cliente':
        if str(v_dash or '').lower() == str(v_sheet or '').lower(): return False
    if campo == 'observacion':
        if str(v_dash or '').strip() == str(v_sheet or '').strip(): return False # ignorar solo-espacio
    return True
```

Criterio de "% prom." en Urgentes (23/07/2026)

La tarjeta General y las solapas por GA usan el MISMO criterio:

% prom = (días prom - 3) / 3 (promedio de días SIN redondear). Verde = adelanto,
rojo = retraso. En pedidosGeneralSummary: _demTot = (_dpRaw - 3) / 3 con
_dpRaw = avg(días_prod no null). NO afecta al % Servicio, que usa avgDem
(demora de entrega, con estimados para vencidos sin fecha real).

Días prom. / % prom.: solo entregados (29/07/2026)

Días prod. prom., % prom. y avgDemDP se calculan **solo sobre pedidos ENTREGADOS**
(con fecha_entrega_real); los pedidos en proceso NO cuentan. Si un GA/condición no tiene
entregados, la vista muestra "—". Así no se muestra adelanto/retraso de artículos aún en
fabricación. Helpers en el dashboard:

```
function entregadoP(p){return p.fecha_entrega_real&p.fecha_entrega_real!='-';}
function dpProm(arr){return avg(arr.filter(entregadoP).map(p=>p.dias_prod));}
// avgDemDP filtra además por entregadoP(p)
```